idox mobile: 1B model (MiniCPM-V 4.6)

On-device PDF-to-Word conversion · Samsung Galaxy M35 (SM-M356B, Exynos 1380) · CPU only · as of 2026-10-12

1. Summary

Item	Result
Digital PDFs (text mode)	**100/100 accuracy** on both test documents, **57–77 s** per 2-page document
Digital PDFs (image mode, before text mode)	280–326 s per 2-page document; small word and structure errors
Peak memory	1.56–1.75 GB (lowest of the tested models)
Not yet done	Scanned-PDF runs, and runs on the new dot-product libraries (both were done on the 2B only)

2. Configuration

Setting	Value
Model files	`minicpm-v-4.6-q4_k_m.gguf` + projector `mmproj-minicpm-v-4.6-q8_0.gguf` (728 MB)
Runtime	llama.cpp `llama-server` bundled in the APK as `libllama-server.so`, started from `nativeLibraryDir` with `LD_LIBRARY_PATH`
Server options	`-t 4 -tb 4 -Cr 4-7 -Crb 4-7 --cpu-strict 1 --cpu-strict-batch 1 -fa on -np 1 -c 8192 --jinja -ngl 0 --no-mmproj-offload --no-op-offload`
Image tokens	Fixed at 512 via `LLAMA_ARG_IMAGE_MIN_TOKENS` / `LLAMA_ARG_IMAGE_MAX_TOKENS`
KV cache	Default 16-bit (8-bit KV was applied to the 2B only)
Request path	`/apply-template` then streamed `/completion` with a top-level `json_schema` (page schema) or a grammar
Decoding	temperature 0; `repeat_penalty` 1.0 in text mode, 1.15 in image mode
App	`mobile/idox-mobile` (Java, package `com.idox.mobile`, minSdk 28, targetSdk 35); Word written directly by `DocxWriter` to `Download/idox`

3. Problems faced and how they were solved

#	Problem	Fix
1	Models pushed with adb were not readable by the app	Copied with `run-as` into the app's private storage
2	`/v1/chat/completions` returned HTTP 500 (peg-native parser)	Switched to `/apply-template` + `/completion`
3	MiniCPM's chat template not applied correctly	`--jinja` added for MiniCPM (also LFM2 and Paddle), not for Qwen
4	App killed by Android's low-memory killer during long conversions	Conversion moved to a foreground service (`ConvertService`)
5	Model load time added to the first conversion	Model preloaded when the app opens; summaries report `model_start: warm/cold`
6	The prompt's "write everything" rule caused loops; relaxing it caused early stops	Continuation from the model's own answer (cut back to the last complete block) instead of a retry
7	Answers looped until the token limit	`PageGuard`: stop when a text repeats 3×, or a run of 6 values repeats 3× (catches looping tables); salvage every completed block; drop exact repeats

8	Model copied Markdown marks (#, *, |) from the input into the text	`PageGuard.cleanMarkdown`
9	Structure errors on digital pages (folded headings, wrong block kinds, rows outside tables)	`TextRepair` using the PDF's text layer. 1B results before → after: test doc p1 61/62 words + 6 structure errors → **62/62 exact**; p2 54/56 → **56/56 exact**; digital doc p1 52/54 → **54/54 exact**
10	**Latency: image mode took 280–326 s per document**	**Text mode:** for digital PDFs the text layer is read with PdfBox-Android as Markdown and sent instead of the picture. No image encode, far fewer input tokens. Result: **57–77 s (about 4–5× faster)** with 100/100 accuracy
11	Model kept writing after the page was complete	Block limit (`maxItems` = expected blocks + 1) and a "done" stop: all text-layer words written and a block repeated
12	Charts produced junk text	Prompt rule 7 rewritten: a chart is one image block

4. Results

Latency (2-page documents)

Document	Image mode	Text mode (3 runs)	Peak memory
sample_digital_document.pdf	280.4 s, 322.5 s	60.0 s, 64.7 s, **57.4 s**	1.63–1.75 GB
sample_test_document.pdf	326.1 s, 325.8 s	70.6 s, 77.1 s, **73.3 s**	1.56–1.75 GB

Accuracy (text mode, after TextRepair)

Document	Words	Numbers / table cells	Structure	Score
Digital doc	119/119	all correct	2/2 pages exact	**100**
Test doc	118/118	all correct (20/20 cells per table)	2/2 pages exact	**100**

5. Reference: teammate's MiniCPM-V 4.6 latency findings (PAN cards, separate app, 2026-09-30)

Measured on the same phone model with the same model files, in the teammate's VLM Tester app. Included because the findings apply to the 1B here.

Where the time goes (one PAN card, ~41 s)

Stage	Time
Startup (context, 3 GB KV cache from the default 262,144-token context, projector)	~6.8 s
Vision encode: 3 tiles × ~7.3 s	21.9 s
Image chunks into the LM (3 × ~1.3 s)	3.9 s
Text prefill + 65 generated tokens	6.8 s

Why always 3 tiles

- The model's tile size is 448 px. If either side is over 448 the image is sliced; the slicer never makes one slice, so slicing means at least 2 slices + 1 overview = 3 tiles.
- Each tile is ~60–64 tokens and ~7–8 s to encode. Latency steps with tile count; the only lever is the input picture.

Fixes and experiments

Change	Result

`-c 4096` (instead of the 262,144 default)	About −6 s per scan, output identical. *Our app already sets `-c 8192`, so this saving is already included.*
`-fit off`	No gain
Field canvas: crop the field regions from the original photo into one 448×448 image (1 tile, 64 image tokens)	~22 s instead of ~40 s; 115/125 fields (92%) but only 15/25 documents fully correct. Repeatable digit slips (0001 → 0017). Not ready to ship; needs card detection
OpenMP pinning, GPU, cropping below 448 px	Already disproven

Risk noted: the app's "PAN check: HIGH" only checks format, not correctness; a run with two wrong names passed it.

6. Open items for the 1B

- Run the scanned PDF (stored scan image) on the 1B.
- Re-run all documents on the new dot-product libraries (2× faster reading on the 2B).
- Try 8-bit KV cache on the 1B (saved ~410 MB on the 2B).